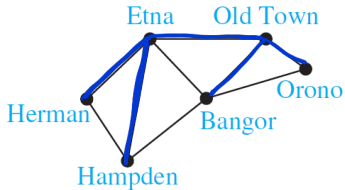


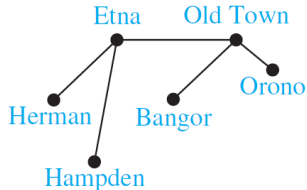
Spanning trees

(Paragraph 11.4)

Let G be a simple graph. A **spanning tree of G** is a subgraph of G that is a tree containing every vertex of G .



(a)



(b)

(a) is a road system

(b) At least five roads must be plowed to ensure that there is a path between any two towns.

Theorem

A simple graph is connected if and only if it has a spanning tree.

There is a path between any two vertices.

Proof:

← A simple graph G has a sp. tr. T .

T has every vertex of G .

T is a tree so there is a path between any two vertices in T .

Since the edges of T are the edges of G there is the path in $G \Rightarrow G$ is connected.

→ G is a simple connected graph

If G is not a tree it has a simple circuit. Remove one edge of the circuit to get subgraph which is connected and has all vertices of G .

if this subgraph is a tree $\Rightarrow$ spanning tree
if it is not a tree then again take a circuit and repeat the process until we get a tree.



Breadth-First search

Algorithm: Breadth-First Search

```
procedure BFS ( $G$  : connected graph with vertices  $v_1, v_2, \dots, v_n$ )
```

$$T := \text{tree consisting only of vertex } v_1$$
$$L := \text{empty list}$$

put v_1 in the list L of unprocessed vertices

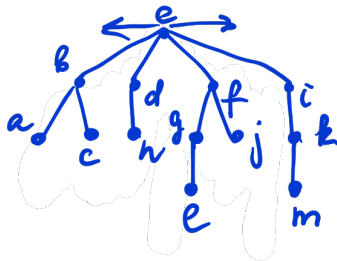
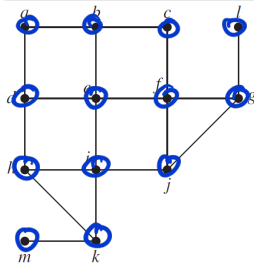
while L is not empty

remove the first vertex, v , from L

for each neighbor w of v

if w is not in L and not in T then

add w to the end of the list L , add w and edge (v, w) to T



The complexity is $O(n^2)$.

Depth-First search

This is a backtracking algorithm

Algorithm: Depth-First Search

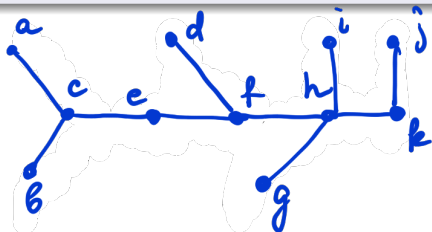
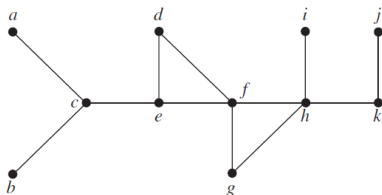
procedure DFS(G : connected graph with vertices $v_1, v_2, \dots, v_n$)

$T :=$ tree consisting only of the vertex v_1

visit(v_1)

procedure visit(v : vertex of G)

for each vertex w adjacent to v and not yet in T add vertex w , edge (v, w) to T
visit(w)



The complexity is $O(n^2)$.

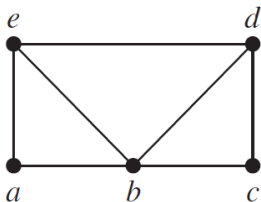
Answer Q3

Backtracking applications

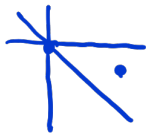
Some problems can be solved only by performing an exhaustive search of all possible solutions. One way to search systematically for a solution is to use a decision tree, where each internal vertex represents a decision and each leaf a possible solution.

To find a solution via **backtracking**, first make a sequence of decisions in an attempt to reach a solution as long as this is possible. The sequence of decisions can be represented by a path in the decision tree. Once it is known that no solution can result from any further sequence of decisions, **backtrack** to the parent of the current vertex and work toward a solution with another series of decisions, if this is possible. The procedure continues until a solution is found, or it is established that no solution exists.

Graph Colorings: Backtracking to decide whether a graph can be colored using 3 colors?

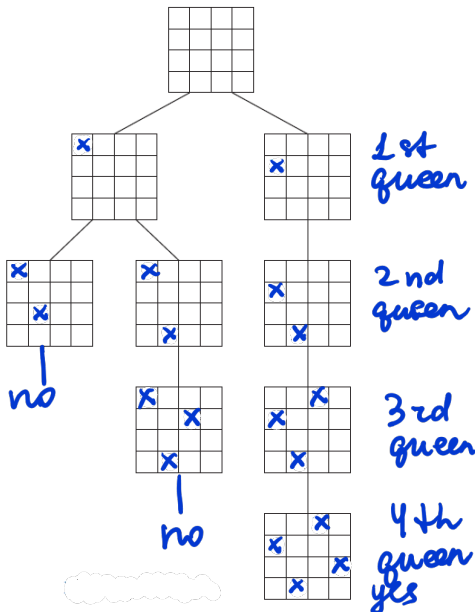


The n -Queens problem



The n -queens problem asks how n queens can be placed on an $n \times n$ chessboard so that no two queens can attack one another. How can backtracking be used to solve this problem?

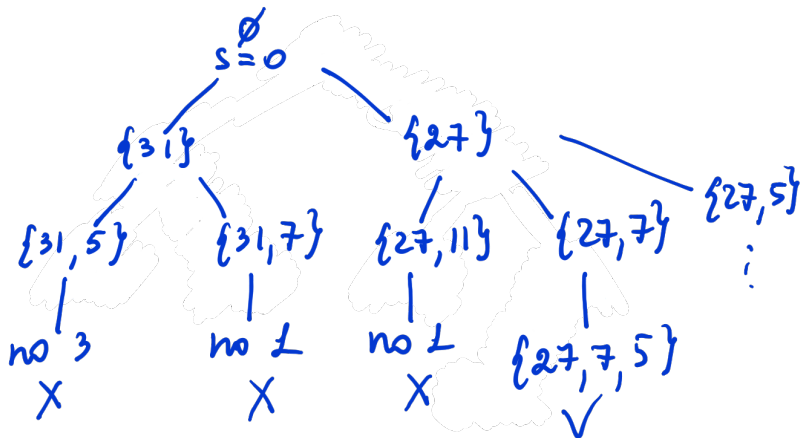
For example take $\underline{n = 4}$ we have



Sum of subsets

Given a set of positive integers $x_1, x_2, \dots, x_n$, find a subset of this set of integers that has M as its sum. How can backtracking be used to solve this problem?

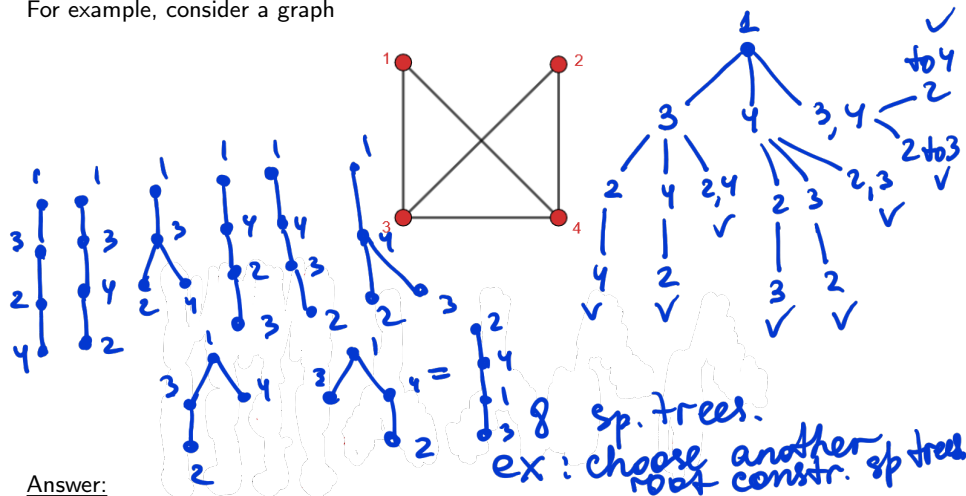
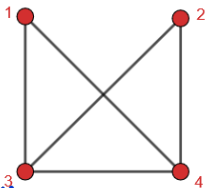
For example, find a subset of $\{31, 27, 15, 11, 7, 5\}$ with the sum equal to 39.



Total number of spanning trees

How many spanning trees has a simple graph? How can backtracking be used to solve this problem?

For example, consider a graph



Answer:

Number of spanning trees

Algorithm from [here](#)

Follow the given procedure :-

STEP 1: Create Adjacency Matrix for the given graph.

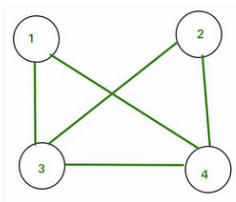
STEP 2: Replace all the diagonal elements with the degree of nodes. For eg. element at (1,1) position of adjacency matrix will be replaced by the degree of node 1, element at (2,2) position of adjacency matrix will be replaced by the degree of node 2, and so on.

STEP 3: Replace all non-diagonal 1's with -1.

STEP 4: Calculate co-factor for any element.

STEP 5: The cofactor that you get is the total number of spanning tree for that graph.

Consider the following graph:



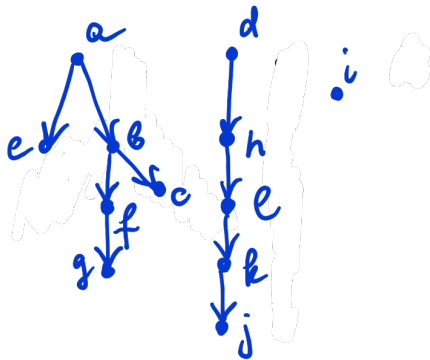
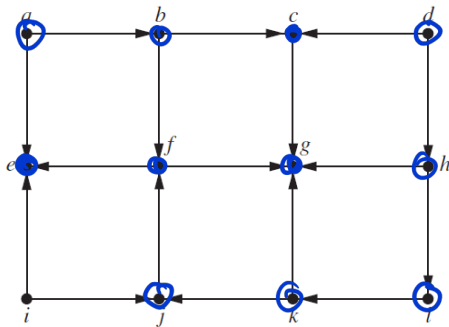
$$\begin{bmatrix} 2 & 0 & -1 & -1 \\ 0 & 2 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{bmatrix}$$
$$18 - 1 - 1 - 3 - 3 - 2 = 8$$

Answer Q4

Depth-First search in directed graphs

We can easily modify both depth-first search and breadth-first search so that they can run given a directed graph as input.

- The output will not necessarily be a spanning tree, but rather a spanning forest.
- In both algorithms we add an edge only when it is directed away from the vertex.
- The next vertex added by the algorithm becomes the root of a new tree in the spanning forest.



Web spiders

The search engines (Google, Yahoo, etc.) use programs called Web spiders (or crawlers or bots) to visit websites and analyze their contents.

The World Wide Web can be modeled as a directed graph where each Web page is represented by a vertex and where an edge starts at the Web page a and ends at the Web page b if there is a link on a pointing to b .

Web spiders use both depth-first searching and breadth-first searching.

Using depth-first search, an initial Web page is selected, a link is followed to a second Web page (if there is such a link), a link on the second Web page is followed to a third Web page, if there is such a link, and so on, until a page with no new links is found. Backtracking is then used to examine links at the previous level to look for new links, and so on.

Using breadth-first search, an initial Web page is selected and a link on this page is followed to a second Web page, then a second link on the initial page is followed (if it exists), and so on, until all links of the initial page have been followed.

Homework:

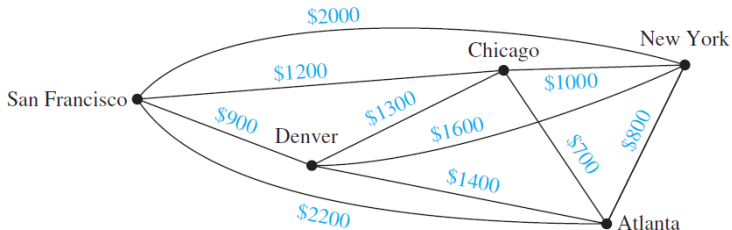
- Solve odd exercises until #29 starting at p. 795.

Minimum spanning trees

(Paragraph 11.5)

A graph is called **weighted** if there is a weight (cost) for each edge.

A weighted graph showing monthly lease costs for lines in a computer network



A **minimum spanning tree** in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.

There are two algorithms **Prim's algorithm** and **Kruskal's algorithm** for finding minimum spanning tree.

Prim's algorithm

The first algorithm that we will discuss was originally discovered by the Czech mathematician [Vojtěch Jarník](#) in 1930, who described it in a paper in an obscure Czech journal.

The algorithm became well known when it was rediscovered in 1957 by [Robert Prim](#).

Because of this, it is known as Prim's algorithm (and sometimes as the Prim-Jarník algorithm).

- Begin by choosing any edge with smallest weight, putting it into the spanning tree.
- Add to the tree edges of minimum weight that are incident to a vertex already in the tree.
- Never forming a simple circuit with those edges already in the tree.
- Stop when $n - 1$ edges have been added.

Prim's Algorithm

procedure Prim(G : weighted connected undirected graph with n vertices)

$T :=$ a minimum-weight edge

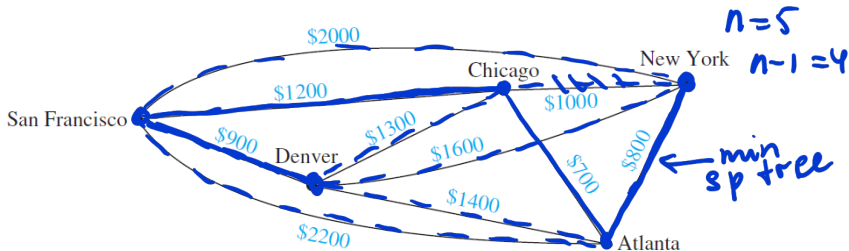
for $i := 1$ **to** $n - 2$

$e :=$ an edge of minimum weight incident to a vertex in T and not forming a simple circuit in T if added to T .

$T := T$ with e added

return T (T is a minimum spanning tree of G)

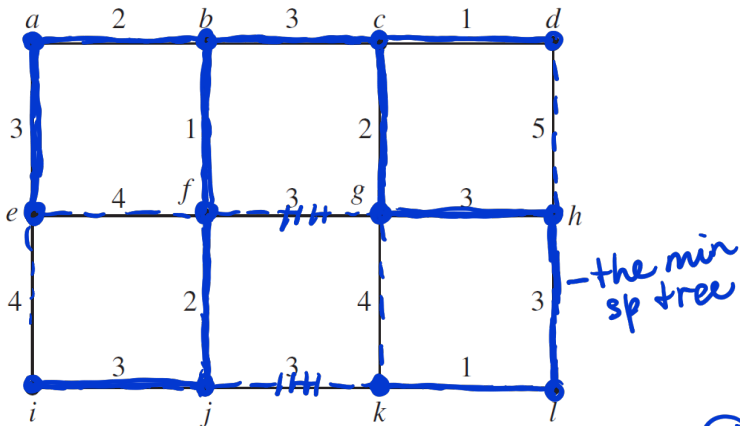
Find a minimum spanning tree using Prim's algorithm of the weighted graph



Answer: (Chicago, Atlanta), (Atlanta, New York) (Chicago, San Francisco), (San Francisco, Denver) with cost: $700+800+1200+900=$ 3600.

Example

Find a minimum spanning tree using Prim's algorithm of the weighted graph. What is the minimal cost?



Answer:

$$1 + 3 + 3 + 2 + 1 + 3 + 2 + 1 + 3 + 2 + 3 = 15 + 3 + 6 = 24$$

Answer Q5

Theorem

Prim's algorithm produces a minimum spanning tree of a connected weighted graph.

Proof:

Assume that Prim's alg. gives a tree S with edges $e_1, e_2, \dots, e_{n-1}$.

Let T is the min. sp. tree of G .

We will show that $T=S$.

k - max. number of common edges of S and T .

Need to show that $k=n-1$.

Assume that $k < n-1 \Rightarrow e_1, e_2, \dots, e_k \in T$

Consider graph $T \cup \{e_{k+1}\}$ but $e_{k+1} \notin T$.

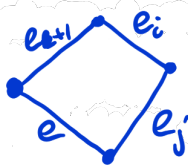
This graph has n edges (too many to be a tree)

$\Rightarrow$ There is a simple circuit, which must contain e_{k+1}

S_{k+1} is tree with $e_1, e_2, \dots, e_{k+1}$

In ~~S_{k+1}~~ there is must be an edge which is not in S_{k+1}
the simple circuit

Proof(continuation):



e is not in S_{k+1}
Remove e from T and add e_{k+1}
Obtain a new tree T'

T' has $e_1, e_2, e_3, \dots, e_k, e_{k+1}$

$|e_{k+1}| \leq |e| \Rightarrow T'$ is also a min sp. tree.

$\Rightarrow$ this means that T' (min sp. tree) and S have $k+1$ common edges.

But k is the max number of comm. edges of S and min sp. tree.

$\Rightarrow$ Contradiction! $\Rightarrow k = n-1 \Rightarrow S = 1$

$\Rightarrow$ Prim's alg give a min sp. tree



Kruskal's algorithm

The second algorithm we will discuss was discovered by Joseph Kruskal in 1956.

- Choose an edge in the graph with minimum weight
- add edges with minimum weight that do not form a simple circuit with those edges already chosen
- Stop after ~~$n-1$~~ ^{$n-1$} edges have been selected

Kruskal's Algorithm

procedure Kruskal(G : weighted connected undirected graph with n vertices)

$T :=$ empty graph

for $i := 1$ **to** $n - 1$

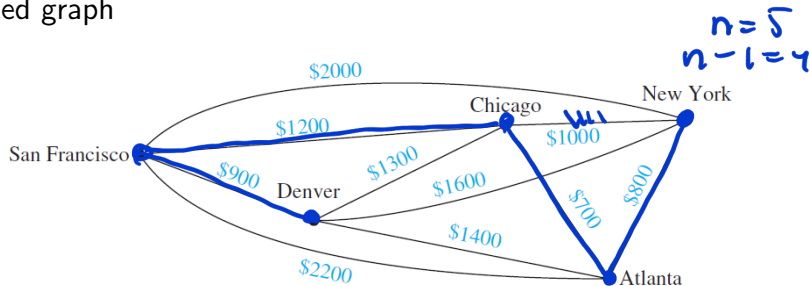
$e :=$ an edge with smallest weight not forming a simple circuit in T if added to T .

$T := T$ with e added

return T (T is a minimum spanning tree of G)

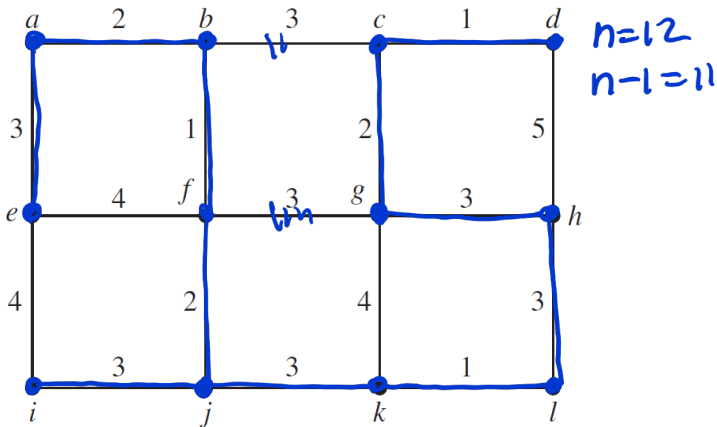
Examples

Find a minimum spanning tree using Kruskal's algorithm of the weighted graph



Answer: (Chicago, Atlanta), (Atlanta, New York) (Chicago, San Francisco), (San Francisco, Denver) with cost: $700+800+1200+900=3600$.

Find a minimum spanning tree using Kruskal's algorithm of the weighted graph. What is the minimal cost?



Answer: $3 \cdot 1 + 3 \cdot 2 + 5 \cdot 3 = 3 + 6 + 15 = 24$

Answer Q6

to find a minimum spanning tree of a graph with m edges and n vertices

- using Prim's algorithm can be carried out using $O(m \log n)$ operations
- using Kruskal's algorithm can be carried out using $O(m \log m)$ operations

If $m < n(n-1)/2$ (the total number of possible edges in an undirected graph with n vertices), then it is more efficient to use Kruskal's algorithm.

Otherwise, there is little difference in the complexity of these two algorithms.

Homework:

- Solve odd exercises until #25 starting at p. 802.